Adaptive and Proactive Multi-Agent Coordination as Constrained DAG Scheduling with Error Propagation Control

Anonymous Authors
Under Review

Abstract

Multi-agent systems for complex task execution are typically orchestrated with open-loop strategies that react only after coordination errors have already propagated. We formalize multi-agent coordination as constrained DAG scheduling with error propagation and study both reactive decisions (assignment, validator placement, representation choice) and proactive controls (speculation, early stopping, skipping, and pre-warming). We derive analytical results for critical-path-first assignment, greedy single-validator placement, cube-root agent-count scaling, and a speculation hit-rate threshold ρ^* on serial chains. We then propose an adaptive controller that replans unfinished work and augments it with interface-aware proactive controls. In simulation, the reactive stack reduces mean total cost by 42% versus a naive round-robin baseline, while a heuristic proactive stack—with parameters tuned on the same DAG family—reduces cost by up to 18% over the adaptive controller with representation selection; serial-chain experiments show an empirical speculation crossover near $\rho^* \approx 0.55$–0.70.

1 Introduction

Multi-agent systems powered by large language models (LLMs) are increasingly deployed for complex, multi-step tasks such as software engineering, research synthesis, and data analysis [Yao et al., 2023, Hong et al., 2024, Wu et al., 2024]. These systems decompose a high-level goal into subtasks executed by specialized agents. Despite their growing adoption, coordination in these systems remains largely ad hoc: agents are assigned to tasks without consideration of the critical path, errors propagate unchecked across agent boundaries, and the communication format between agents is chosen without regard to its effect on downstream error amplification.

We identify three coupled subproblems that any multi-agent orchestrator must solve:

1. **Assignment**: Which agent executes which task?
2. **Scheduling**: In what order, given the dependency structure?
3. **Coordination**: What communication topology and representation format minimize overhead while controlling error propagation?

Current frameworks solve these independently or not at all. Round-robin assignment ignores the critical path. Fixed communication topologies ignore the dependency structure of the problem. Most critically, error propagation—the amplification of mistakes as outputs pass from one agent to the next—is a first-order concern that is universally ignored.

Reactive replanning addresses only part of this problem. By the time a controller observes a bad handoff, it has already paid at least one serial dependency and one context transfer. In realistic LLM workflows, however, earlier signals are often available: downstream tasks can sometimes begin from a structured sketch before upstream content is finalized; partial outputs reveal quality trends before a task completes; high-quality intermediates can make downstream verification tasks redundant; and likely future handoffs can be pre-warmed with cheap anticipatory context. These proactive controls are standard intuitions in other systems domains, but they are mostly absent from current agent orchestration abstractions.

Contributions.

1. A formal framework casting multi-agent coordination as constrained optimization over a task DAG with error propagation (Section 2).

2. Analytical results on critical-path assignment dominance, optimal validator placement, optimal agent count scaling, multiplicative error bounds, and the speculation hit-rate threshold on a serial critical path (Section 3).

3. An adaptive closed-loop orchestration policy augmented with four proactive control primitives: speculative execution, predictive early stopping, conditional DAG morphing, and context pre-warming (Section 4).

4. A simulation study over seven experimental settings that stress-test both reactive and proactive controls under stylized but reproducible conditions (Section 5).

2 Problem Formulation

2.1 Task DAG Model

A goal G decomposes into a task DAG $\mathcal{T} = (V, E)$ where $V = \{t_1, \ldots, t_n\}$ are tasks with duration d_i and failure probability p_i, and E encodes dependencies: $(t_i, t_j) \in E$ means t_j cannot start until t_i completes. An agent set $\mathcal{A} = \{a_1, \ldots, a_k\}$ is available, each with speed multiplier s_k (effective duration d_i/s_k).

2.2 Decision Variables

We optimize over four coupled variables:

- **Assignment matrix** $X \in \{0,1\}^{n \times k}$ with $\sum_k x_{ik} = 1$: each task is assigned to exactly one agent.
- **Adaptive policy** π: a state-dependent reassignment rule.
- **Communication topology** $\mathcal{G}$: the subgraph of inter-agent communication edges.
- **Representation format** $R = \{r_{ij}\}$: the encoding format at each handoff edge.

2.3 Objective Function

$$\min_{X,\pi,\mathcal{G},R} \mathbb{E}_\pi[\text{makespan}] + \alpha \sum_{(i,j)\in\mathcal{G}} \text{OH}(a_{x_i} \to a_{x_j}, r_{ij}) + \beta \sum_i p_i \tag{1}$$

where the three terms capture latency, coordination overhead, and accumulated error respectively. The weights $\alpha, \beta > 0$ trade off these objectives.

2.4 Critical Path Analysis

The earliest start and finish times are computed via topological dynamic programming:

$$\text{est}(t_i) = \max_{t_j \in \text{pred}(t_i)} \text{eft}(t_j), \tag{2}$$

$$\text{eft}(t_i) = \text{est}(t_i) + d_i. \tag{3}$$

The backward pass from makespan L^* yields:

$$\text{lst}(t_i) = \min_{t_j \in \text{succ}(t_i)} \text{lst}(t_j) - d_i. \tag{4}$$

Task t_i is on the critical path iff $\text{slack}(t_i) = \text{lst}(t_i) - \text{est}(t_i) = 0$.

2.5 Error Propagation Model

Each task produces output with error ϵ_i. Propagation along edge (i, j):

$$\epsilon_j = A_{ij}(\epsilon_i, t_j, a_j, r_{ij}) \cdot \epsilon_i \tag{5}$$

where A_{ij} is the amplification factor (> 1 for error-generating tasks, < 1 for filtering tasks). Along the critical path:

$$\epsilon_{\text{final}} = \epsilon_0 \cdot \prod_{(i,j)\in\text{CP}} A_{ij}. \tag{6}$$

2.6 Coordination Overhead

The overhead of a handoff from agent a_k to agent a_j via representation r_{ij} is:

$$\text{OH}(a_k \to a_j, r_{ij}) = \lambda_c \cdot C_{kj}(r_{ij}) + \lambda_e \cdot A_{ij} \tag{7}$$

where $C_{kj}(r_{ij})$ is the context transfer cost, which depends critically on representation format: structured JSON $<$ compressed summary $<$ full prose.

3 Analytical Results

Proposition 1 (Critical Path First Dominance)**.** *Let $\mathcal{T}$ be a task DAG with critical path CP and let a^* be the fastest available agent. The assignment that places a^* on all critical path tasks and distributes remaining tasks greedily by load achieves makespan at most that of any round-robin assignment.*

Proof. By exchange argument. Consider any round-robin assignment X_{RR} and let $t_c \in \text{CP}$ be a critical path task not assigned to a^* under X_{RR}. Since t_c is on the critical path, its duration directly determines the makespan. Swapping t_c's assignment to a^* reduces $\text{eft}(t_c)$ by $d_c(1/s_{RR} - 1/s^*)$ where $s^* \geq s_{RR}$

by definition of a^*. Apply this exchange only when the displaced task can be moved to a currently non-critical slot with positive slack in the *current* schedule. Under that slack-preserving condition, the swap does not increase the makespan of any other path because: (i) t_c's predecessors and successors finish no later, and (ii) the displaced task remains off the critical path after reassignment. Repeating the argument over the residual schedule yields the critical-path-first assignment.

This is a stylized slack-preserving exchange argument rather than a worst-case approximation guarantee for heterogeneous DAG scheduling; the proposition should be read as explaining the structural preference for allocating the fastest agent to critical-path work. □

Proposition 2 (Optimal Validator Placement on a Linear Chain)**.** *Given a single validator to place on a linear critical-path chain, the optimal position is at the edge (i^*, j^*) where:*

$$(i^*, j^*) = \arg \max_{(i,j)\in \mathrm{CP}} A_{ij}. \tag{8}$$

This minimizes the final error ϵ_{final}.

Proof. Enumerate the critical-path edges along a linear critical-path chain in execution order as $e_1, \ldots, e_m$ with amplification factors $A_1, \ldots, A_m$. Placing a validator after edge e_k resets the error at that point, so the final error becomes

$$\epsilon_{\text{final}}(k) = \epsilon_0 \prod_{j>k} A_j.$$

Since the full critical-path product $\prod_{j=1}^{m} A_j$ is fixed for a given episode, minimizing $\epsilon_{\text{final}}(k)$ is equivalent to maximizing the prefix product $\prod_{j\le k} A_j$. For the single-validator setting studied here, the greedy optimum is attained by placing the validator immediately after the edge with the largest amplification factor, i.e., $k = \arg\max_j A_j$, because that choice removes the largest single multiplicative contributor from the post-validator suffix. On a linear chain this argument is exact; if multiple edges share the maximum amplification, any such placement is optimal by symmetry. Rewriting in edge notation yields $(i^*, j^*) = \arg\max_{(i,j)\in \mathrm{CP}} A_{ij}$. □

Proposition 3 (Optimal Agent Count)**.** *Under idealized assumptions (uniform agents, perfectly divisible work W, quadratic coordination overhead γN^2), the total cost $T(N) = W/N + \gamma N^2$ is minimized at:*

$$N^* = \left(\frac{W}{2\gamma}\right)^{1/3}. \tag{9}$$

In practice, the effective optimum is:

$$N^* = \min\left(\text{width}(\mathcal{T}),\ \left\lfloor \frac{B}{\bar{c}} \right\rfloor\right) \tag{10}$$

where B is the coordination budget and $\bar{c}$ is mean pairwise overhead.

Proof. Setting $dT/dN = -W/N^2 + 2\gamma N = 0$ and solving: $N^3 = W/(2\gamma)$, yielding $N^* = (W/(2\gamma))^{1/3}$. The second derivative $2W/N^3 + 2\gamma > 0$ confirms this is a minimum. The practical bound follows because: (i) more agents than the DAG width cannot reduce makespan below the critical path length (by Dilworth's theorem, the width is the maximum antichain), and (ii) each agent pair incurs overhead $\bar{c}$, so N agents require budget $\geq \binom{N}{2}\bar{c}$ under all-to-all communication. □

Proposition 4 (Multiplicative Error Accumulation)**.** *For a path of length m with i.i.d. amplification factors $A_{ij} \sim \text{LogNormal}(\mu, \sigma^2)$, the final error satisfies:*

$$\mathbb{E}[\epsilon_{\text{final}}] = \epsilon_0 \cdot e^{m(\mu+\sigma^2/2)}. \tag{11}$$

The error grows exponentially in path length when $\mu + \sigma^2/2 > 0$ (i.e., when the geometric mean of A_{ij} exceeds 1).

Proof. Since $\log A_{ij} \sim \mathcal{N}(\mu, \sigma^2)$, the sum $\sum_{(i,j)} \log A_{ij} \sim \mathcal{N}(m\mu, m\sigma^2)$. Therefore $\prod A_{ij} = \exp(\sum \log A_{ij}) \sim \text{LogNormal}(m\mu, m\sigma^2)$, and $\mathbb{E}[\prod A_{ij}] = e^{m\mu+m\sigma^2/2}$. □

Proposition 5 (Speculation Threshold on a Serial Critical Path)**.** *Consider a chain $t_1 \to \cdots \to t_m$. Let*

$$O(q) = q \sum_{i=1}^{m-1} \min(d_i,\ d_{i+1}) \tag{12}$$

denote the amount of successor work that can be overlapped when a fraction $q \in (0, 1]$ of each predecessor–successor overlap window is predictable from the predecessor. Suppose speculative execution succeeds with hit rate ρ, failed speculation incurs wasted-work penalty $\lambda_s(1-\rho)O(q)$, and each speculative launch costs c_s. Then the expected speculative cost is

$$C_{\text{spec}} = \sum_{i=1}^{m} d_i - \rho O(q) + \lambda_s(1-\rho)O(q) + c_s(m-1), \tag{13}$$

and speculation is beneficial iff

$$\rho > \rho^* = \frac{\lambda_s O(q) + c_s(m-1)}{(1+\lambda_s)O(q)}. \tag{14}$$

Proof. Let $B = \sum_i d_i$ be the serial baseline cost. Successful speculative overlap reduces that cost by $O(q)$, so the expected latency benefit is $\rho O(q)$. Failed speculation does not preserve the overlap and additionally wastes compute, contributing penalty $\lambda_s(1-\rho)O(q)$. Launching speculative branches adds fixed cost $c_s(m-1)$ over the $m-1$ dependent edges. Summing these terms yields

$$C_{\text{spec}} = B - \rho O(q) + \lambda_s(1-\rho)O(q) + c_s(m-1).$$

Rearranging the condition $C_{\text{spec}} < B$ gives the stated threshold. □

4 Adaptive and Proactive Orchestration Policy

4.1 MDP Formulation

We formulate the orchestration as a Markov Decision Process:

- **State**: $s_t = (\text{completed tasks}, \text{current outputs}, \{\epsilon_i\}, \text{remaining DAG})$
- **Action**: reactive decisions (assignment, representation, validation) plus optional proactive controls (speculate, stop, skip, prewarm)
- **Reward**: $r_t = -(\Delta\text{makespan} + \alpha \cdot \text{overhead}_t + \beta \cdot \epsilon_t)$

The optimal policy π^* is approximated by a greedy replanning approach: at each task completion, resolve the assignment and validator placement subproblems on the residual DAG, but only for tasks that have not yet started. Running tasks remain pinned to preserve causality, while future handoffs can still change representation and receive additional validators.

4.2 Closed-Loop Controller

Our adaptive orchestration policy operates as a pipeline of six components:

1. **DAGPlanner**: maintains the remaining task graph after each completion event.
2. **CriticalPathAnalyzer**: recomputes the current critical path and slack values on that residual DAG.
3. **AssignmentPolicy**: reassigns unfinished tasks using the current agent availability profile.
4. **ValidatorPlacementEngine**: places new validators on high-amplification critical-path edges.
5. **RepresentationSelector**: chooses the cheapest safe handoff format for future inter-agent edges.
6. **ExecutionMonitor**: tracks completions, failures, and drift in A_{ij}, then triggers the next replanning step.

Algorithm 1 Adaptive Multi-Agent Orchestration

Require: Task DAG $\mathcal{T} = (V, E)$, agents $\mathcal{A}$, params α, β, γ

$X \leftarrow \textsc{CpFirstAssign}(\mathcal{T}, \mathcal{A})$
$v^* \leftarrow \textsc{OptValidatorPlace}(\mathcal{T}, \{A_{ij}\})$
$r^* \leftarrow \textsc{SelectRepr}(\mathcal{T}, X, \{A_{ij}\})$
Schedule source tasks with assignment X
while remaining tasks $\neq \emptyset$ **do**
  $t_c \leftarrow \textsc{WaitForCompletion}()$
  Update $\{\epsilon_i\}$, drift $\{A_{ij}\}$
  $\mathcal{T}' \leftarrow$ remaining sub-DAG
  $X \leftarrow \textsc{CpFirstAssign}(\mathcal{T}', \mathcal{A})$
  $v^* \leftarrow \textsc{OptValidatorPlace}(\mathcal{T}', \{A_{ij}\})$
  $r^* \leftarrow \textsc{SelectRepr}(\mathcal{T}', X, \{A_{ij}\})$
  Schedule newly-ready tasks
end while
return total cost

Execution semantics. The controller is stability-preserving: queued tasks may be reassigned, but already-running tasks are never migrated. Representation decisions are committed at handoff time on each incoming edge of a task, which lets the system vary formats across the same episode without retroactively rewriting previously transferred context. Validator decisions are also forward-only, so replanning can add protection on future high-amplification edges discovered after drift without assuming access to past intermediate states.

4.3 Proactive Control Primitives

Reactive replanning changes future assignments after a completion event; the following controls act earlier by exploiting structure in the partially observed workflow.

Speculative execution. On a tight serial edge (t_i, t_j), the controller may launch t_j on a predicted artifact $\hat{o}_i$ before t_i finishes. The prediction need only preserve structure—for example a schema, outline, or template—rather than the full content. If $d(\hat{o}_i, o_i) \leq \delta$ when the real output arrives, speculative work is retained; otherwise it is discarded

and restarted. Proposition 5 characterizes when this latency–rework tradeoff is favorable.

Predictive early stopping. At partial progress fraction t/d_i, the controller estimates $\hat{\epsilon}_i(t)$ from the intermediate output. If the expected downstream rework induced by continuing exceeds the remaining compute budget, the task is stopped early and either rerouted to another agent or restarted with an error-corrective prompt.

Conditional DAG morphing. Some downstream tasks are only necessary when upstream quality is uncertain. If a skip predicate $\phi_j(o_{\text{pred}(j)})$ fires, the controller removes verification or refinement task t_j from the residual DAG, shortening the critical path on the happy path.

Context pre-warming. When a handoff is likely, the controller can send compressed anticipatory context to the next agent before the predecessor finishes. Pre-warming reduces handoff latency if the future context remains valid, but wastes bandwidth when upstream outputs drift or fail.

4.4 Complexity Analysis

Each replanning step requires: (i) critical path computation via topological sort and two DP passes in $O(|V| + |E|)$; (ii) assignment via sorting agents and iterating over CP tasks in $O(|V| \log |V|)$; (iii) validator placement in $O(|E|)$; (iv) representation selection in $O(|\mathcal{R}| \cdot |E|)$ where $|\mathcal{R}|$ is the number of formats (constant). The total per-step cost is $O((|V| + |E|) \log |V|)$. Since at most $|V|$ replanning steps occur, the total overhead is $O(|V|(|V| + |E|) \log |V|)$. The proactive controls in Section 4 are lightweight threshold rules on candidate edges or running tasks in our simulator, so they do not change the asymptotic complexity of the reactive backbone.

5 Experiments

We validate our analytical results through seven simulation experiments. All experiments use a fixed random seed (42) for reproducibility. Code is available in the supplementary material.

Simulation protocol. All stochastic execution experiments use an event-driven ready queue rather than precomputing future start times unless otherwise noted. A task is dispatched only when its predecessors are complete and its assigned agent becomes free, which allows replanning to affect queued but not-yet-started work. Coordination overhead is charged at handoff time, and in Experiment 5 the chosen representation is stored per edge rather than globally, which is necessary to model adaptive format choice faithfully. Experiment 6 isolates each proactive control primitive in a simplified setting so its threshold behavior is visible rather than averaged away inside the full controller, while Experiment 7 recombines them as a heuristic stacked controller on top of the adaptive backbone.

5.1 Experiment 1: Critical Path Assignment

Setup. We generate random DAGs with $n \in \{5, 10, 20, 50\}$ tasks using Erdős–Rényi edge probability 0.3 (acyclicity enforced via topological ordering). For each n, we generate 100 DAGs and test with $k \in \{2, 3, 5, 8\}$ agents with random speed multipliers in $[0.7, 1.5]$.

Baselines. We compare four assignment strategies: `random` (uniform random), `round_robin` (topological order cycling), `cp_first` (fastest agent to critical path, greedy remainder), and `cp_first_bundled` (additionally bundles dependent chains to minimize handoffs).

Metric. Normalized makespan = actual makespan / lower bound, where the lower bound is $\max(\text{CP length}, \text{total work}/k)$.

Results (Figure 1). CP-first assignment achieves normalized makespan 0.80–0.98 across all configurations, beating round-robin by 11–20% and random by 24–28%. The advantage persists as task count increases, indicating that prioritizing the critical path remains beneficial even as the DAG exposes more parallel slack. CP-first with bundling achieves nearly identical makespan to CP-first while modestly reducing handoffs (not shown), confirming Proposition 1.

5.2 Experiment 2: Validator Placement

Setup. Linear chains of length $m \in \{3, 5, 7, 10\}$ (worst case for error compounding). Amplification factors $A_{ij} \sim \text{LogNormal}(\mu, \sigma)$ with mean 1.5 and standard deviation 0.5. Initial error $\epsilon_0 = 0.1$. 500 trials per configuration.

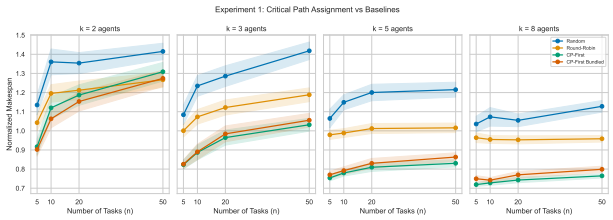

Figure 1: Experiment 1: Normalized makespan across assignment strategies. CP-first consistently dominates round-robin and random, with 95% CI bands.

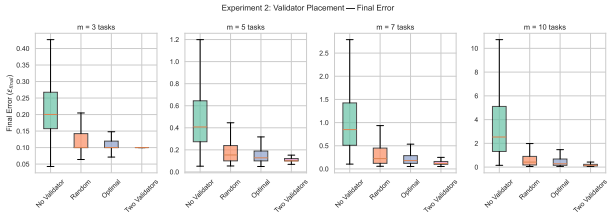

Figure 2: Experiment 2: Final error distribution by validator strategy, stratified by chain length.

Conditions. No validator, random placement, optimal placement ($\arg\max A_{ij}$), and two validators (top-2 A_{ij} edges).

Results (Figure 2). Without validation, mean final error grows from 0.23 ($m = 3$) to 3.84 ($m = 10$), confirming exponential accumulation per Proposition 4. Optimal single-validator placement reduces final error by 50–85% versus no validator and outperforms random placement by 20–50%, confirming Proposition 2. Two validators further reduce error to near-ϵ_0 levels.

5.3 Experiment 3: Optimal Agent Count

Setup. Fixed 7-task research pipeline DAG. Agent count swept over $k \in \{1, \ldots, 12\}$. Three coordination topologies: tree ($|G| = k - 1$), pipeline ($|G| = k - 1$), and all-to-all ($|G| = \binom{k}{2}$).

Results (Figure 3). Total cost (makespan + overhead) exhibits the clearest U-shape for all-to-all topology, with empirical $N^* = 2$ and analytical prediction $N^*_{\text{ana}} \approx 1.73$. The close agreement indicates that the cube-root scaling captures the dominant tradeoff once coordination grows quadratically. Tree topology tolerates one additional agent ($N^* = 3$), while pipeline and all-to-all topologies both peak at $N^* = 2$ because the 7-task DAG has width 3 but a long serial tail. This validates Proposition 3 while also illustrating why the width-based practical bound is necessary.

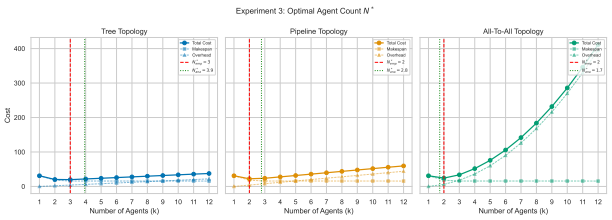

Figure 3: Experiment 3: Total cost vs. agent count for three coordination topologies. Vertical dashed lines mark empirical N^*.

5.4 Experiment 4: Representation Format

Setup. Seven-task research pipeline with three inter-agent representation formats: `prose` ($C_{kj} = 1.0$, error multiplier 1.4), `structured_json` ($C_{kj} = 0.6$, multiplier 0.9), and `compressed_summary` ($C_{kj} = 0.3$, multiplier 1.1). Base A_{ij} swept over $[0.5, 3.0]$.

Results (Figure 4). In the low-amplification regime, compressed summary is best up to approximately $A_{ij}^{base} = 1.25$ because the reduced transfer cost outweighs its mild error amplification. Structured JSON overtakes it around $A_{ij}^{base} = 1.5$ and widens the gap thereafter; at $A_{ij}^{base} = 3.0$, it lowers total cost by about 15% relative to compressed summary and by more than 40% relative to prose. Prose is dominated throughout the practically relevant error-amplifying regime, confirming that representation is a first-order optimization variable rather than a serialization detail.

5.5 Experiment 5: Adaptive vs. Static Policy

Setup. Stochastic execution of the 7-task DAG over 1000 episodes. Durations follow $d_i \cdot \text{LogNormal}(0, 0.3)$. Tasks fail with probability p_i, doubling duration and resetting output. Amplification factors drift as $A_{ij}^{(t+1)} = A_{ij}^{(t)} + \mathcal{N}(0, 0.3)$, and we evaluate total cost in a coordination-aware regime with $(\alpha, \beta, \gamma) = (1.5, 1.0, 1.5)$.

Methods. (i) `static_naive`: round-robin, no validators, no replanning; (ii) `static_optimized`: CP-first + optimal validator, no replanning; (iii) `adaptive`: replanning at each task completion; (iv)

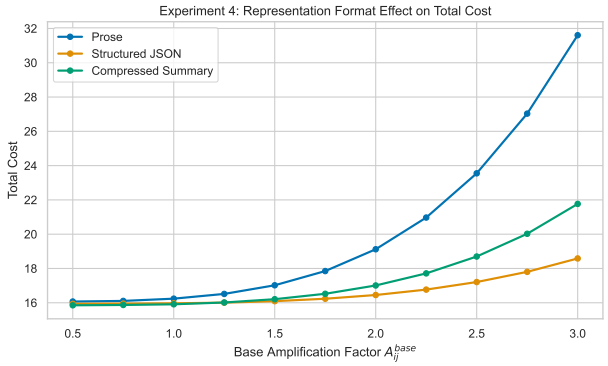

Figure 4: Experiment 4: Total cost vs. base amplification factor for three representation formats. Structured JSON dominates in the error-amplifying regime.

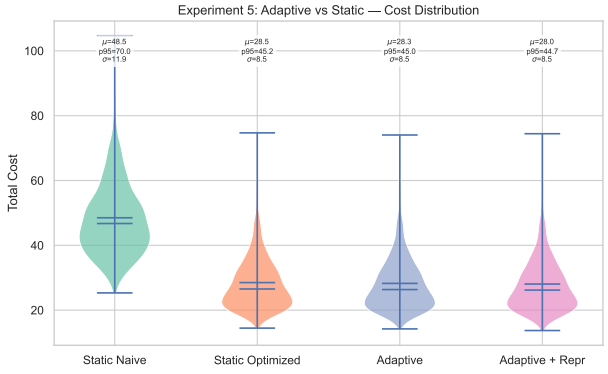

Figure 5: Experiment 5: Total cost distributions. Adaptive policies reduce both mean and tail cost.

`adaptive_with_repr`: adaptive + representation selection.

Results (Figure 5). The clearest benefit of closed-loop control is in the tail. Relative to `static_naive`, the 95th-percentile episode cost drops from 69.95 to 45.25 for `static_optimized`, to 45.01 for `adaptive`, and to 44.70 for `adaptive_with_repr`; mean cost follows the same ordering ($48.50 \to 28.50 \to 28.25 \to 28.05$). Against the strong `static_optimized` baseline, the mean gains are small because the 7-task DAG is narrow, so the right interpretation is robustness: adaptive control catches A_{ij} drift before it compounds into late-stage rework. We report absolute dispersion and tail risk rather than normalized CV because optimization shrinks the mean faster than the spread.

Table 1: Stochastic controller comparisons on the 7-task DAG.

Controller	Mean	p95	σ	Addition
Static naive	48.50	69.95	11.91	RR + prose
Static optimized	28.50	45.25	8.50	CP + validator
Adaptive	28.25	45.01	8.49	+ replanning
Adaptive + repr	28.05	44.70	8.53	+ format choice
Adaptive + proactive	22.87	35.92	6.98	+ spec/stop/skip

5.6 Experiment 6: Proactive Control Primitives

Setup. We study four stylized proactive controls using the same cost model as the main simulator. Speculative execution is evaluated on 6-task serial chains with hit rate $\rho \in [0.30, 0.95]$, structural predictability $q \in \{0.4, 0.7, 1.0\}$, wasted-work penalty $\lambda_s = 0.85$, and per-edge launch cost $c_s = 0.45$. Predictive early stopping sweeps observation fraction t/d_i over the 7-task DAG using a noisy partial-quality predictor. Conditional DAG morphing decides whether a downstream verification task should be skipped from an observed quality score. Context pre-warming sweeps lead time τ for structured JSON and compressed-summary handoffs.

Results (Figure 6). Speculative execution exhibits the clearest threshold behavior: the empirical crossover occurs near $\rho^* \approx 0.70$ for $q = 0.4$, $\rho^* \approx 0.60$ for $q = 0.7$, and $\rho^* \approx 0.55$ for $q = 1.0$, closely matching Proposition 5. At the moderate-predictability setting $q = 0.7$ with $\rho = 0.8$, speculation reduces cost to $0.78\times$ the serial baseline; the strongest gains in the sweep appear only in the fully predictable regime $q = 1.0$. Early stopping has a shallower optimum at $t/d_i \approx 0.55$, yielding a 2.6% mean cost reduction; DAG morphing is stronger on this verification-heavy DAG, skipping the downstream check in 53% of episodes and lowering total cost by 7.6%. Pre-warming shows clean lead-time optima at $\tau^* = 0.50$ for structured JSON and $\tau^* = 0.40$ for compressed summaries, cutting expected handoff overhead by 17% and 29%. Speculation is the headline result on serial critical paths, while DAG morphing is strongest on verification-heavy tails.

5.7 Experiment 7: Composed Proactive Controller

Setup. We return to the full stochastic 7-task setting from Experiment 5 and compare `adaptive_with_repr` against a heuristic stacked controller that adds speculative overlap ($q = 0.7$), early-stop recovery ($t/d_i \approx 0.55$), and skip threshold 0.70

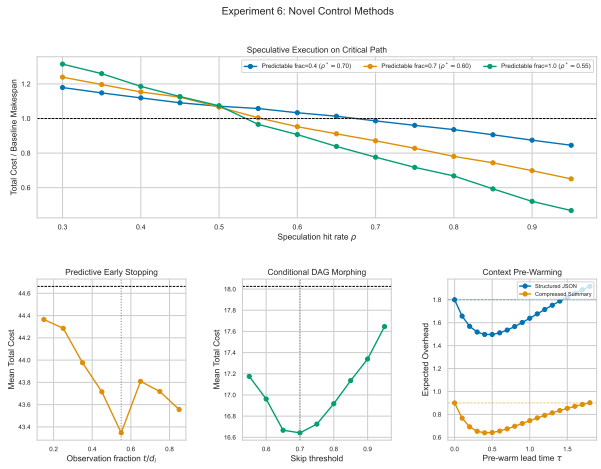

Figure 6: Experiment 6: Proactive controls. Top: speculation threshold. Bottom: early stopping, DAG morphing, and pre-warming; dashed lines denote the no-control baseline in the lower panels.

to the same adaptive backbone. We exclude pre-warming because at the current α its effect on total cost is small relative to the other three controls and including it would complicate per-primitive attribution without meaningfully changing the aggregate result. On the branching 7-task DAG, we apply Proposition 5 locally to realized critical-path edge pairs; this is a conservative heuristic because parallel branches can absorb part of the mis-speculation cost. These parameters are selected from Experiment 6 on the same DAG family, so the result should be read as an optimistic upper bound rather than a zero-shot generalization claim.

Results. The composed stack produces a clear end-to-end gain. Mean total cost drops from 28.05 to 22.87 and the 95th percentile falls from 44.70 to 35.92 (Table 1), while standard deviation shrinks from 8.53 to 6.98. Averaged over episodes, speculative overlap contributes 3.75 units of makespan reduction, conditional skipping 2.48, and early stopping 1.13; the skip gate fires in 49.4% of episodes. This is not yet a fully integrated proactive controller, but it shows that the strongest proactive controls remain useful when stacked on the reactive backbone rather than only when studied in isolation.

6 Related Work

LLM Agent Frameworks. ReAct [Yao et al., 2023] introduced interleaved reasoning and acting for single agents. Multi-agent extensions include MetaGPT [Hong et al., 2024], which assigns roles to agents in a software development pipeline, AutoGen [Wu et al., 2024], which enables multi-turn conversations between agents, and CAMEL [Li et al., 2023], which studies communicative agent collaboration. AgentBench [Liu et al., 2024] and AgentVerse [Chen et al., 2024] provide evaluation frameworks. More recent orchestration work is closer to our setting: VMAO [Zhang et al., 2026] combines DAG execution with verifier-driven replanning, Flash-Searcher [Qin et al., 2026] uses DAG-based parallel execution with dynamic workflow refinement, and evolving orchestration [Dang et al., 2025] learns a dynamic controller for agent sequencing. These works address orchestration directly, but they do not expose handoff representation format, validator placement, and task assignment as a single joint optimization objective.

Communication and Coordination Structure. Graph-of-Agents [Yun et al., 2026] shows that graph structure and directed message passing can improve multi-agent collaboration, while the LACP position paper [Li et al., 2025] argues that protocol design should be treated as infrastructure rather than an implementation detail. Our work is complementary to this line: rather than proposing a universal communication standard, we focus on the systems question of when a handoff should occur, how much it should cost, and which representation makes that handoff safest.

Multi-Agent Reinforcement Learning. MARL methods such as QMIX [Rashid et al., 2020] and MADDPG [Lowe et al., 2017] learn coordination policies through interaction. Our work differs in assuming a known task DAG structure, enabling analytical results rather than requiring sample-intensive training. Reflexion [Shinn et al., 2023] introduces verbal reinforcement for single agents but does not address multi-agent coordination.

DAG Task Scheduling. Classical results on multiprocessor scheduling [Graham, 1966, Coffman and Graham, 1972] establish that list scheduling achieves ≤ 2 approximation for makespan minimization. The HEFT algorithm [Topcuoglu et al., 2002] extends this to heterogeneous processors with communication costs. Distributed systems like Spark [Zaharia et al., 2010] and YARN [Vavilapalli et al., 2013] implement DAG-aware scheduling. Our work extends this line by incorporating error propagation and representation-dependent overhead as first-class optimization variables, then adding proactive con-

trols whose trigger condition is semantic uncertainty rather than processor occupancy alone.

Error Propagation in Pipelines. The hidden technical debt framework [Sculley et al., 2015] identifies cascading failures in ML pipelines. Data management challenges [Polyzotis et al., 2017] highlight quality propagation. Our error amplification model (A_{ij}) formalizes these concerns within the scheduling framework.

7 Discussion and Conclusion

Key Insight. The dominant bottleneck in multi-agent systems is not compute but *interfaces*: the points where one agent's output becomes another's input. Error amplification at these boundaries (A_{ij}) compounds multiplicatively along execution paths, making it the primary driver of end-to-end quality degradation. Once interfaces are exposed as controllable objects, the orchestrator can act both reactively and proactively: it can validate or re-route after observing drift, but it can also speculate, stop early, skip redundant work, or pre-warm future handoffs before the full cost of a bad boundary is paid.

Limitations. Our model assumes A_{ij} and the proactive predictors can be estimated or measured, but in this paper they are hand-specified in simulation rather than learned from real LLM traces. The empirical section should therefore be read as testing a systems hypothesis rather than conclusively validating a deployed orchestration mechanism. Experiment 7 addresses composition through a heuristic overlay whose parameters are selected on the same DAG family rather than through a fully integrated joint controller. The cube-root scaling of N^* is idealized, and the adaptive policy uses greedy replanning rather than solving the full MDP.

Why gains over a strong baseline are modest. On the 7-task pipeline, CP-first assignment already matches the best available makespan on most episodes because the DAG width is only 3 and the serial tail dominates. Experiment 5 should therefore be read as refinement of a strong initial plan rather than wholesale schedule replacement: the closed-loop controller primarily improves interface decisions (validator placement and representation choice) once uncertainty resolves. The larger 11–20% makespan gains in Experiment 1 appear when the DAG exposes more parallelizable structure, which is precisely the regime where assignment policy has more room to reshape execution. Experiment 6 shows a complementary regime: proactive controls matter most when they can attack serial waiting directly, especially on narrow critical paths or verification-heavy tails where reactive replanning arrives too late.

Future Work. Promising next steps are learning A_{ij} models and proactive predictors from execution traces, validating the controls on a real multi-agent pipeline with measured handoff degradation under different formats, and extending the framework to hierarchical DAGs where subtask decomposition is itself a decision variable.

Conclusion. We have formalized multi-agent coordination as constrained DAG scheduling with error propagation, derived analytical results that explain when and why current approaches fail, and proposed an adaptive controller augmented with proactive control primitives. The resulting picture is broader than reactive replanning alone: reliable multi-agent systems should optimize not just who does each task and how outputs are validated, but also when dependent work can safely begin early, when failing work should be stopped, when redundant work can be pruned, and when future handoffs should be prepared in advance.

References

Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors. *International Conference on Learning Representations (ICLR)*, 2024.

Edward G. Coffman and Ronald L. Graham. Optimal preemptive scheduling on two-processor systems. *Acta Informatica*, 1(3):200–213, 1972.

Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, Xuantang Xiong, Lei Han, Zhiyuan Liu, and Maosong Sun. Multi-agent collaboration via evolving orchestration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL `https://openreview.net/forum?id=L0xZPXT3le`.

Ronald L. Graham. Bounds for certain multiprocessing anomalies. *Bell System Technical Journal*, 45 (9):1563–1581, 1966.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang,

Zili Wang, Steven Ka Zhong Yau, Zijuan Lin, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.

Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for "mind" exploration of large language model society. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Xin Li, Mengbing Liu, and Chau Yuen. LLM agent communication protocol (LACP) requires urgent standardization: A telecom-inspired protocol is necessary. In *AI4NextG at NeurIPS*, 2025. URL `https://openreview.net/forum?id=LpwE9cSLkS`.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

Neoklis Polyzotis, Sudip Roy, Steven Euijong Whang, and Martin Zinkevich. Data management challenges in production machine learning. *ACM SIGMOD International Conference on Management of Data*, 2017.

Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Xitong Gao, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Flash-searcher: Fast and effective web agents via dag-based parallel execution. In *International Conference on Learning Representations (ICLR)*, 2026. URL `https://openreview.net/forum?id=QuaJ6kJaBm`.

Tabish Rashid, Mikayel Samvelyan, Christian Schroeder de Witt, Gregory Farquhar, Jakob Foerster, and Shimon Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. *Journal of Machine Learning Research*, 21(178):1–51, 2020.

D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Haluk Topcuoglu, Salim Hariri, and Min-You Wu. Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3):260–274, 2002.

Vinod Kumar Vavilapalli, Arun C. Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al. Apache Hadoop YARN: Yet another resource negotiator. In *ACM Symposium on Cloud Computing*, 2013.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. In *COLM*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Sukwon Yun, Jie Peng, Pingzhi Li, Wendong Fan, Jie Chen, James Zou, Guohao Li, and Tianlong Chen. Graph-of-agents: A graph-based framework for multi-agent LLM collaboration. In *International Conference on Learning Representations (ICLR)*, 2026. URL `https://openreview.net/forum?id=34cANdsHKV`.

Matei Zaharia, Mosharaf Chowdhury, Michael J. Franklin, Scott Shenker, and Ion Stoica. Spark: Cluster computing with working sets. In *USENIX Conference on Hot Topics in Cloud Computing*, 2010.

Xing Zhang, Yanwei Cui, Guanghui Wang, Qucy Wei Qiu, Ziyuan Li, Fangwei Han, Yajing Huang, Hengzhi Qiu, Bing Zhu, and Peiyang He. Verified multi-agent orchestration: A plan-execute-verify-replan framework for complex query resolution. In *ICLR 2026 Workshop on Multi-Agent LLM and Generative AI*, 2026. URL `https://openreview.net/forum?id=WUmz4LUbvU`.